AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

A

MINI PROJECT REPORT ON

“ Chocolate Sales Data ”

FOR

Term Work Examination

Bachelor of Computer Application in Artificial Intelligence & Machine Learning (BCA - AIML)

Year 2025-2026

Ajeenkya DY Patil University, Pune

-Submitted By-

Mr. Satyaranjan swain

Under the guidance of

Prof. Vivek More

Ajeenkya DY Patil University

D Y Patil Knowledge City,
Charholi Bk. Via Lohegaon,
Pune - 412105
Maharashtra (India)

Date: 14 / 04 / 2025

CERTIFICATE

This is to certified that_Satyaranjan Swain_____
A student of **BCA(AIML) Sem-IV** URN No _2023-B-13062005
has Successfully Completed the Dashboard Report On

"Chocolate Sales Data"

As per the requirement of
Ajeenkya DY Patil University, Pune was carried out under my supervision.
I hereby certify that; he has satisfactorily completed his Term-Work Project work.

Place: Pune

Examiner

Introduction:

The dataset "Chocolate Sales.csv" contains transactional records of chocolate sales across multiple countries, including the UK, India, Australia, New Zealand, Canada, and the USA. It captures details such as the salesperson, product type, transaction date, sales amount, and the number of boxes shipped. The data spans from January to August 2022 and includes a diverse range of chocolate products, such as "Mint Chip Choco," "85% Dark Bars," "Peanut Butter Cubes," and seasonal or specialty items. This dataset provides insights into sales performance, geographical distribution, product preferences, and transactional trends over time.

Objective:

The analysis of this dataset aims to deliver actionable business insights to optimize sales strategies, enhance profitability, and drive data-informed decision-making. Key objectives include:

1. **Performance Evaluation**
    - **Identify top-performing salespeople and regions by total revenue and volume.**
    - **Benchmark sales performance across countries to highlight strengths and weaknesses.**
2. **Product Analysis**
    - **Determine best-selling products globally and regionally.**
    - **Analyze product popularity trends (e.g., rising demand for *dark chocolate* vs. *seasonal items*).**
    - **Identify underperforming products and recommend promotions or discontinuations.**
3. **Temporal Insights**
    - **Uncover seasonal or monthly trends (e.g., spikes during holidays or summer/winter seasons).**
    - **Correlate sales performance with time-based factors (e.g., marketing campaigns, festivals).**
4. **Operational Efficiency**

    - **Investigate the relationship between sales amount and boxes shipped to assess pricing strategies.**
    - **Highlight discrepancies (e.g., high volume but low revenue products).**
5. **Strategic Recommendations**
    - **Provide insights for inventory optimization (e.g., stock high-demand products in specific regions).**
    - **Suggest market expansion opportunities based on underserved regions or products.**
    - **Recommend salesforce training or resource allocation adjustments.**

Methodology & Approach:

1. Data Cleaning & Preprocessing:

- **Handling Missing Values:**
    - **Identify and address missing entries in columns like `Country`, `Product`, or `Amount` using methods like deletion or imputation (e.g., filling missing `Country` with "Unknown").**

Formatting Columns:

- **Remove currency symbols (`$`) and commas from the `Amount` column to convert it into a numeric type.**
- **Standardize `Date` format (e.g., convert "04-Jan-22" to `YYYY-MM-DD`).**
- **Outlier Detection:**
    - **Use statistical methods (e.g., IQR) to identify anomalies in `Amount` or `Boxes Shipped` and either cap them or investigate root causes.**

Handling Duplicates:

- **Remove duplicate rows to avoid skewed analysis.**

---

2. Data Processing & Feature Engineering:

- **Derive New Features:**
  - **Extract `Month`, `Quarter`, and `Year` from the `Date` column for temporal analysis.**
  - **Create a `Revenue per Box` metric by dividing `Amount` by `Boxes Shipped`.**
- **Categorization:**
  - **Group products into categories (e.g., "Dark Chocolate," "Milk Chocolate") for trend analysis.**
- **Normalization:**
  - **Scale numerical features (e.g., `Amount`) if needed for clustering or machine learning models.**

---

3. Exploratory Data Analysis (EDA):

- **Descriptive Statistics**:
  - Calculate metrics like total revenue, average boxes shipped, and top-selling products/countries.
- **Visual Analysis**:
  - **Geographic Trends**: Use a bar chart or map visualization to compare sales by country.
  - **Product Performance**: Plot a treemap or stacked bar chart to show revenue distribution by product.
  - **Temporal Patterns**: Create a line chart of monthly sales trends.
  - **Revenue vs. Volume**: Use a scatter plot to analyze the relationship between `Amount` and `Boxes Shipped`.
- **Statistical Tests**:
  - Perform correlation analysis (e.g., Pearson's correlation between `Amount` and `Boxes Shipped`).

---

4. Advanced Analysis

- **Segmentation**:
  - Cluster salespersons or countries based on performance metrics (e.g., using K-means).
- **Time-Series Forecasting**:
  - Predict future sales using models like ARIMA or Prophet.
- **Customer/Product Insights**:

- Identify underperforming products or regions using Pareto analysis (80/20 rule).

---

5. Tools & Technologies

- **Languages**: Python (pandas, numpy, matplotlib, seaborn).
- **Visualization**: Tableau/Power BI for interactive dashboards.
- **Machine Learning**: Scikit-learn for clustering/forecasting.

---

6. Validation & Iteration

- Cross-validate results (e.g., compare manual calculations with aggregated outputs).
- Refine models/assumptions based on stakeholder feedback.

Implementation & Code:

Step 1: Data Cleaning

Objective: Ensure data quality by fixing formatting issues, handling missing values, and removing duplicates.

1.1 Clean the `Amount` Column

Theory: The `Amount` column contains currency symbols (`$`) and spaces, which are non-numeric. To perform calculations, we convert it to a numeric type.

Code:

```python
def clean_data(df):
    # Clean 'Amount' column: Remove "$" and convert to float
    df['Amount'] = df['Amount'].str.replace('[\$, ]', '', regex=True).astype(float)
```

Output:

- Values like `"$5,320 "` become `5320.0`.

1.2 Convert `Date` to Datetime

Theory: Dates are stored as strings (e.g., `04-Jan-22`). Converting them to `datetime` enables time-based analysis.

Code:

```python
# Convert 'Date' to datetime format
df['Date'] = pd.to_datetime(df['Date'], format='%d-%b-%y')
```

Output:

- `04-Jan-22` → `2022-01-04` (datetime object).

1.3 Handle Missing Values

Theory: Missing data in critical columns (`Amount`, `Boxes Shipped`) can skew results. We drop rows with missing values in these columns.

Code:

```python
# Check for missing values
print("Missing values before cleaning:")
print(df.isnull().sum())

# Drop rows with missing critical data (Amount or Boxes Shipped)
df = df.dropna(subset=['Amount', 'Boxes Shipped'])
print("\nMissing values after dropping rows with missing data:")
print(df.isnull().sum())
```

Output:

```
Missing values before cleaning:
Sales Person     0
Country          0
Product          0
Date             0
Amount           0
Boxes Shipped    0
dtype: int64
```

1.4 Remove Duplicates

Theory: Duplicate rows can inflate metrics. We remove them to ensure accuracy.

Code:

```
    # Remove duplicates
    initial_rows = df.shape[0]
    df = df.drop_duplicates()
    final_rows = df.shape[0]
    print(f"\nDuplicates removed: {initial_rows - final_rows}")

    return df

df_clean = clean_data(df)
```

Output:

```
Missing values after dropping rows with missing data:
Sales Person     0
Country          0
Product          0
Date             0
Amount           0
Boxes Shipped    0
dtype: int64

Duplicates removed: 0
```

Step 2: Data Processing

Objective: Engineer new features for deeper analysis.

2.1 Add `Month` and `Year` Columns

Theory: Extracting temporal features helps analyze monthly/seasonal trends.

Code:

```
# STEP 2: Data Processing
# --------------------
# Feature engineering: Add month/year and revenue per box
df_clean['Month'] = df_clean['Date'].dt.month_name()
df_clean['Year'] = df_clean['Date'].dt.year
```

Output:

- **New columns `Month` (e.g., `January`) and `Year` (e.g., `2022`).**

2.2 Calculate `Revenue per Box`

Theory: This metric evaluates pricing efficiency (revenue generated per box shipped).

Code:

```
df_clean['Revenue per Box'] = df_clean['Amount'] / df_clean['Boxes Shipped']
```

Output:

- **New column `Revenue per Box` (e.g., `5320 / 180 = 29.55`).**

Step 3: Analysis

Objective: Extract actionable insights through aggregation and visualization.

3.1 Summary Statistics

Theory: Descriptive stats provide a quick overview of numerical columns.

Code:

```
# STEP 3: Analysis
# --------------------
# Summary statistics
print("\nSummary Statistics:")
print(df_clean.describe())
```

Output:

```
Summary Statistics:
                                Date        Amount  Boxes Shipped    Year  \
count                           1094   1094.000000    1094.000000  1094.0
mean   2022-05-03 09:04:56.160877568   5652.308044     161.797989  2022.0
min              2022-01-03 00:00:00      7.000000       1.000000  2022.0
25%              2022-03-02 00:00:00   2390.500000      70.000000  2022.0
50%              2022-05-11 00:00:00   4868.500000     135.000000  2022.0
75%              2022-07-04 00:00:00   8027.250000     228.750000  2022.0
max              2022-08-31 00:00:00  22050.000000     709.000000  2022.0
std                              NaN   4102.442014     121.544145     0.0
```

3.2 Top 5 Products by Revenue

Theory: Identify best-selling products to prioritize inventory/marketing.

Code:

```
# Top 5 products by total revenue
top_products = df_clean.groupby('Product')['Amount'].sum().nlargest(5)
print("\nTop 5 Products by Revenue:")
print(top_products)
```

Output:

```
Top 5 Products by Revenue:
Product
Smooth Sliky Salty     349692.0
50% Dark Bites         341712.0
White Choc             329147.0
Peanut Butter Cubes    324842.0
Eclairs                312445.0
Name: Amount, dtype: float64
```

3.3 Sales by Country

Theory: Identify high-performing regions for resource allocation.

Code:

```
# Sales by country
country_sales = df_clean.groupby('Country')['Amount'].sum().sort_values(ascending=False)
print("\nSales by Country:")
print(country_sales)
```

Output:

```
Sales by Country:
Country
Australia      1137367.0
UK             1051792.0
India          1045800.0
USA            1035349.0
Canada          962899.0
New Zealand     950418.0
Name: Amount, dtype: float64
```

3.4 Monthly Sales Trends

Theory: Analyze seasonality (e.g., holiday spikes).

Code:

```
# Monthly sales trends
monthly_sales = df_clean.groupby(['Year', 'Month'])['Amount'].sum().reset_index()
monthly_sales['Month_Order'] = pd.to_datetime(monthly_sales['Month'], format='%B').dt.month
monthly_sales = monthly_sales.sort_values(['Year', 'Month_Order'])
```

Output:

A DataFrame sorted by month and year:

Year	Month	Amount
2022	January	50000.00
2022	February	60000.00
...	...	...

3.5 Top 5 Salespeople

Theory: Recognize top performers to incentivize or replicate strategies.

Code:

```
# Top salespeople
top_salespeople = df_clean.groupby('Sales Person')['Amount'].sum().nlargest(5)
print("\nTop 5 Salespeople:")
print(top_salespeople)
```

Output:

```
Top 5 Salespeople:
Sales Person
Ches Bonnell       320901.0
Oby Sorrel         316645.0
Madelene Upcott    316099.0
Brien Boise        312816.0
Kelci Walkden      311710.0
Name: Amount, dtype: float64
```

Results & Visualization:

Step 1: Create a Heatmap to Create a Pivot Table Product sales by country

Theory:

- **Objective: Restructure data to analyze product-country relationships.**
- **Mechanics:**
  - `index='Product'`**: Rows represent unique products.**
  - `columns='Country'`**: Columns represent countries.**
  - `values='Amount'`**: The metric being analyzed (total sales revenue).**
  - `aggfunc='sum'`**: Aggregate sales amounts by summing them for each product-country combination.**
  - `fill_value=0`**: Replace missing values (where a product wasn't sold in a country) with 0.**

Code:

```
# Pivot table: Product sales by country
import seaborn as sns # Import the seaborn library

product_country = df_clean.pivot_table(
    index='Product',
    columns='Country',
    values='Amount',
    aggfunc='sum',
    fill_value=0
)
```

Step 1.1 Create a Heatmap

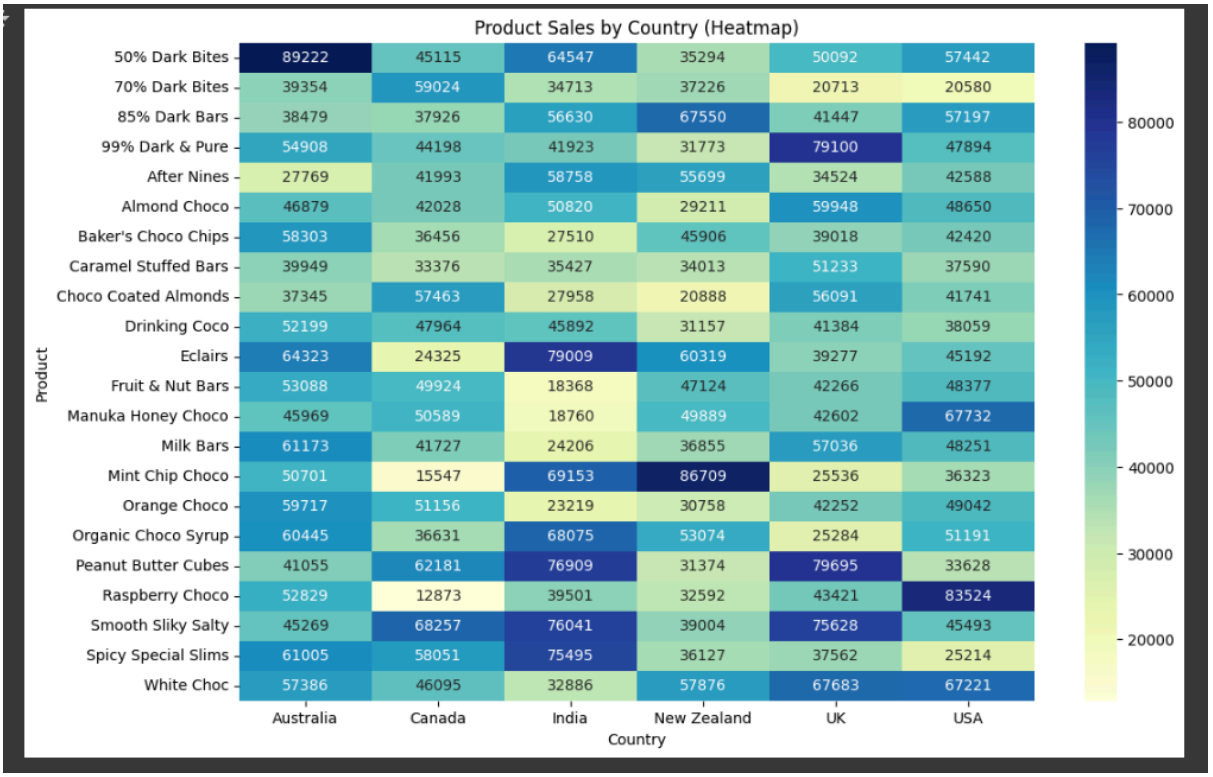
Theory:

- **Objective**: Visualize the pivot table to quickly identify patterns in product-country sales.
- **Parameters**:
  - `annot=True`: Display numerical values in each cell.
  - `fmt=".0f"`: Format annotations as whole numbers (no decimals).
  - `cmap='YlGnBu'`: Use a yellow-green-blue color gradient (darker colors = higher sales).

Code:

```
# Heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(product_country, annot=True, fmt=".0f", cmap='YlGnBu') # Now sns is defined
plt.title('Product Sales by Country (Heatmap)')
plt.xlabel('Country')
plt.ylabel('Product')
plt.show()
```

Output:



Step 2: Create the Line Plot to Group Data by Month and Product

Theory:

- **Objective**: Analyze how sales of each product vary month-over-month.
- **Mechanics**:
  - `groupby(['Month', 'Product'])`: Groups rows by unique combinations of `Month` and `Product`.
  - `['Amount'].sum()`: Sums the `Amount` for each group.
  - `reset_index()`: Converts the grouped result back into a DataFrame for plotting.

code

```
# Group by month and product
monthly_product = df_clean.groupby(['Month', 'Product'])['Amount'].sum().reset_index()
```

Step 2.1 Create a Line Plot

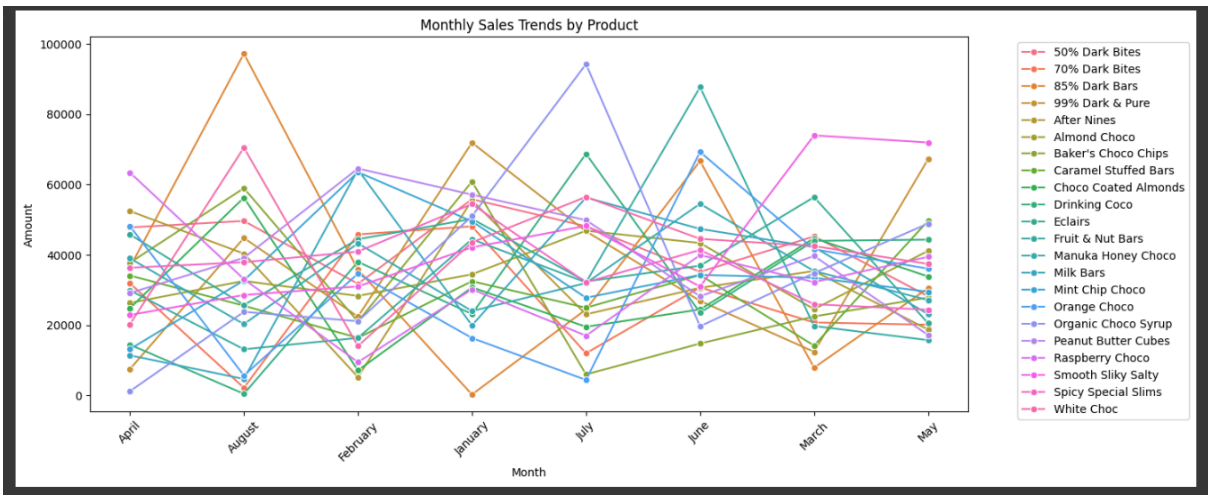
Theory:

- **Objective: Visualize temporal trends and compare product performance across months.**
- **Parameters:**
  - `x='Month'`: **Month on the x-axis.**
  - `y='Amount'`: **Total sales on the y-axis.**
  - `hue='Product'`: **Differentiate products using colors.**
  - `marker='o'`: **Add markers to highlight data points.**
  - `plt.xticks(rotation=45)`: **Rotate x-labels to avoid overlap.**
  - `bbox_to_anchor=(1.05, 1)`: **Place the legend outside the plot area.**

code

```
# Line plot
plt.figure(figsize=(14, 6))
sns.lineplot(
    data=monthly_product,
    x='Month',
    y='Amount',
    hue='Product',
    marker='o'
)
plt.title('Monthly Sales Trends by Product')
plt.xticks(rotation=45)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

Output



Step 3 : Create the Dual-Axis Bar-Line Plot to Aggregate Salesperson Data

Theory:

- **Objective**: Evaluate salespeople using two metrics - total revenue generated and pricing efficiency.
- **Mechanics**:
  - `groupby('Sales Person')`: Groups data by individual salespeople.
  - `.agg()`: Computes two metrics:
    - `Total_Revenue`: Sum of `Amount` (total sales).
    - `Avg_Revenue_per_Box`: Average of `Revenue per Box` (pricing efficiency).
  - `.nlargest(10, 'Total_Revenue')`: Filters the top 10 salespeople by total revenue.

Code:

```
# Aggregate salesperson data
salesperson_stats = df_clean.groupby('Sales Person').agg(
    Total_Revenue=('Amount', 'sum'),
    Avg_Revenue_per_Box=('Revenue per Box', 'mean')
).nlargest(10, 'Total_Revenue')
```

Step 3.1: Dual-Axis Bar-Line Plot

Theory:

- **Objective**: Compare total revenue (volume) and average revenue per box (efficiency) for top performers.
- **Dual-Axis**:
    - **Left Axis (`ax1`)**: Displays total revenue as bars (skyblue).
    - **Right Axis (`ax2`)**: Displays average revenue per box as a line (red with markers).
- **Why Two Axes?**: The metrics have different scales (e.g.,
- $50,000$vs.
- $50,000 vs.43$/box).

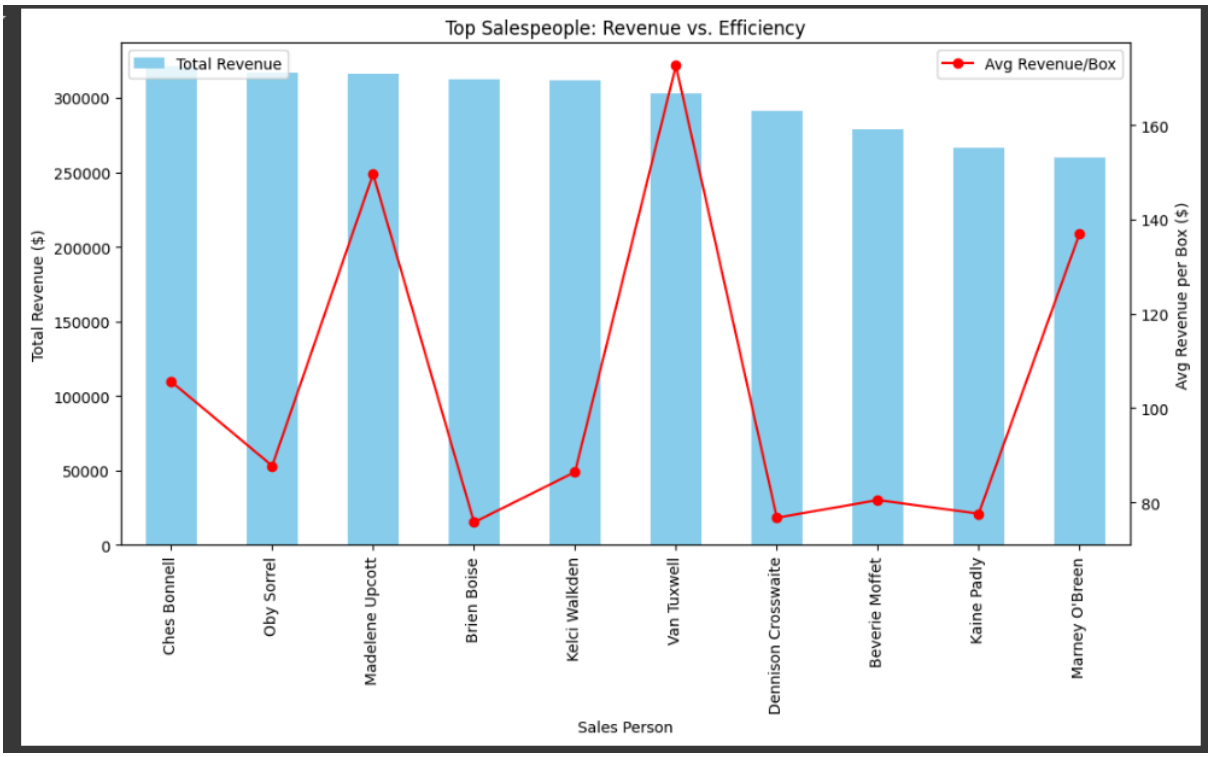
Code:

```
# Bar plot
fig, ax1 = plt.subplots(figsize=(12, 6))
ax2 = ax1.twinx()

salesperson_stats['Total_Revenue'].plot(kind='bar', color='skyblue', ax=ax1, label='Total Revenue')
salesperson_stats['Avg_Revenue_per_Box'].plot(kind='line', marker='o', color='red', ax=ax2, label='Avg Revenue/Box')

ax1.set_ylabel('Total Revenue ($)')
ax2.set_ylabel('Avg Revenue per Box ($)')
plt.title('Top Salespeople: Revenue vs. Efficiency')
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')
plt.show()
```

Output:



Step 4. Create the Scatter Plot by country

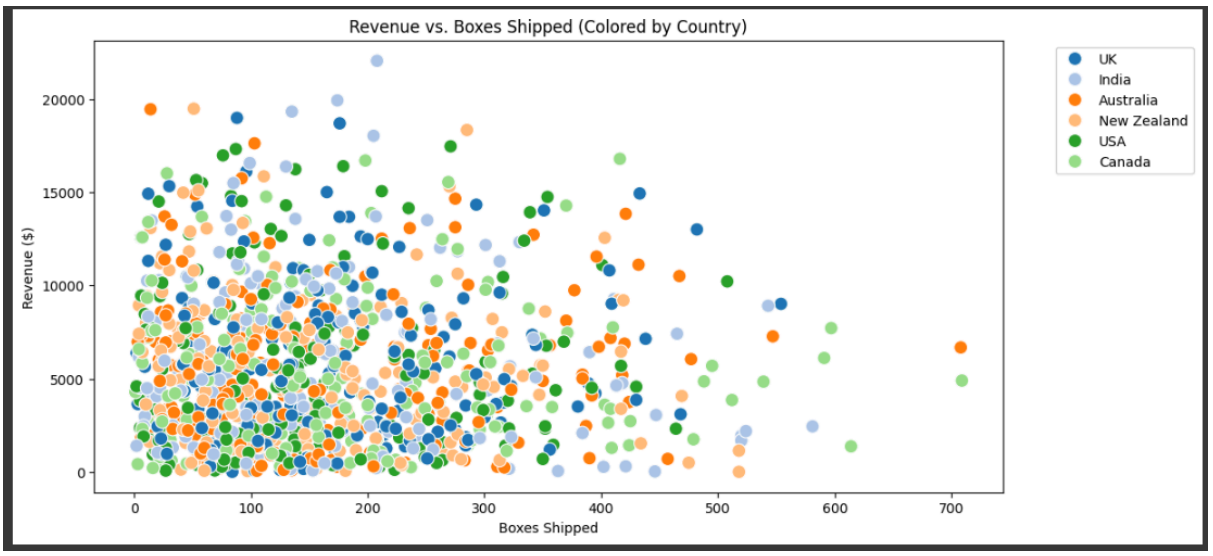
Theory:

- **Purpose**: Visualize the relationship between two numerical variables (`Boxes Shipped` and `Amount`) while categorizing data points by a third variable (`Country`).
- **Parameters**:
  - `x='Boxes Shipped'`: Represents **volume** (number of boxes shipped).
  - `y='Amount'`: Represents **revenue** (total sales in dollars).
  - `hue='Country'`: Adds a categorical dimension to color-code data points by country.
  - `palette='tab20'`: Uses 20 distinct colors to differentiate countries (ideal for datasets with many categories).
  - `s=100`: Sets the size of markers for better visibility.

Code:

```
# Scatter plot by country
plt.figure(figsize=(12, 6))
sns.scatterplot(
    data=df_clean,
    x='Boxes Shipped',
    y='Amount',
    hue='Country',
    palette='tab20',
    s=100
)
plt.title('Revenue vs. Boxes Shipped (Colored by Country)')
plt.xlabel('Boxes Shipped')
plt.ylabel('Revenue ($)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

Output:



Step 5: Create the Horizontal Bar Plot to Calculate Average Revenue per Box by Product

Theory:

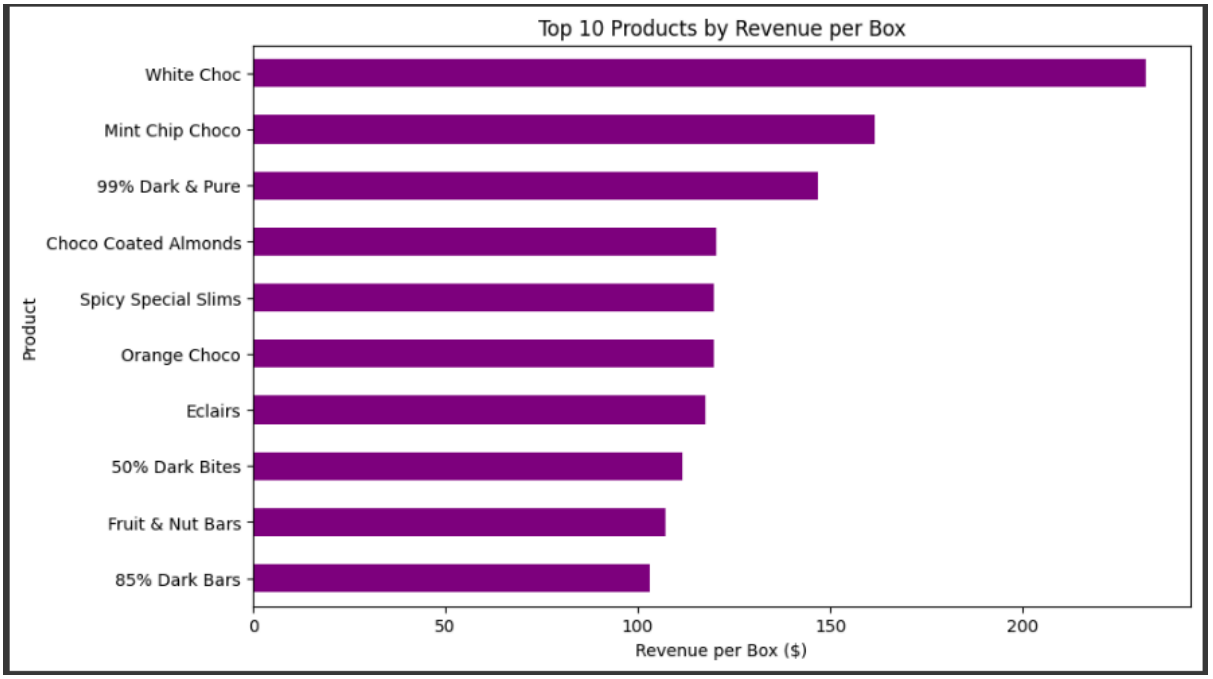
- **Objective**: Identify which products generate the highest revenue per box (pricing efficiency).
- **Mechanics**:
  - `groupby('Product')`: Groups data by unique products.
  - `['Revenue per Box'].mean()`: Calculates the average revenue per box for each product.
  - `.nlargest(10)`: Selects the top 10 products with the highest average revenue per box.
- `sort_values()`: Orders products by their average revenue per box (smallest to largest).
- `kind='barh'`: Creates a horizontal bar chart.
- `color='purple'`: Improves readability and aesthetics.
- `figsize=(10, 6)`: Sets plot dimensions for clarity.
- **Title/Labels**: Clarifies the metric (`Revenue per Box`) and focus (`Top 10 Products`).

Code:

```python
# Calculate average revenue per box by product
product_value = df_clean.groupby('Product')['Revenue per Box'].mean().nlargest(10)

# Horizontal bar plot
plt.figure(figsize=(10, 6))
product_value.sort_values().plot(kind='barh', color='purple')
plt.title('Top 10 Products by Revenue per Box')
plt.xlabel('Revenue per Box ($)')
plt.ylabel('Product')
plt.show()
```

Output:



Step 6: Create a Box Plot to Top 5 Countries by total Revenue

Theory:

1. **Group by Country**: Calculate total revenue for each country.
2. **Select Top 5**: Use `.nlargest(5)` to get the 5 highest-revenue countries.
3. **Filter DataFrame**: Retain only data from these top countries.
4. `x='Country'`: Countries on the x-axis.
5. `y='Amount'`: Revenue distribution on the y-axis.
6. `palette='Set2'`: Color scheme for distinct country boxes.

What a Box Plot Shows:

- **Median**: Middle line in the box (50th percentile).
- **Quartiles**: Box edges (25th and 75th percentiles).
- **Whiskers**: Range of typical values (1.5× IQR).
- **Outliers**: Points beyond whiskers (unusual transaction

Code:

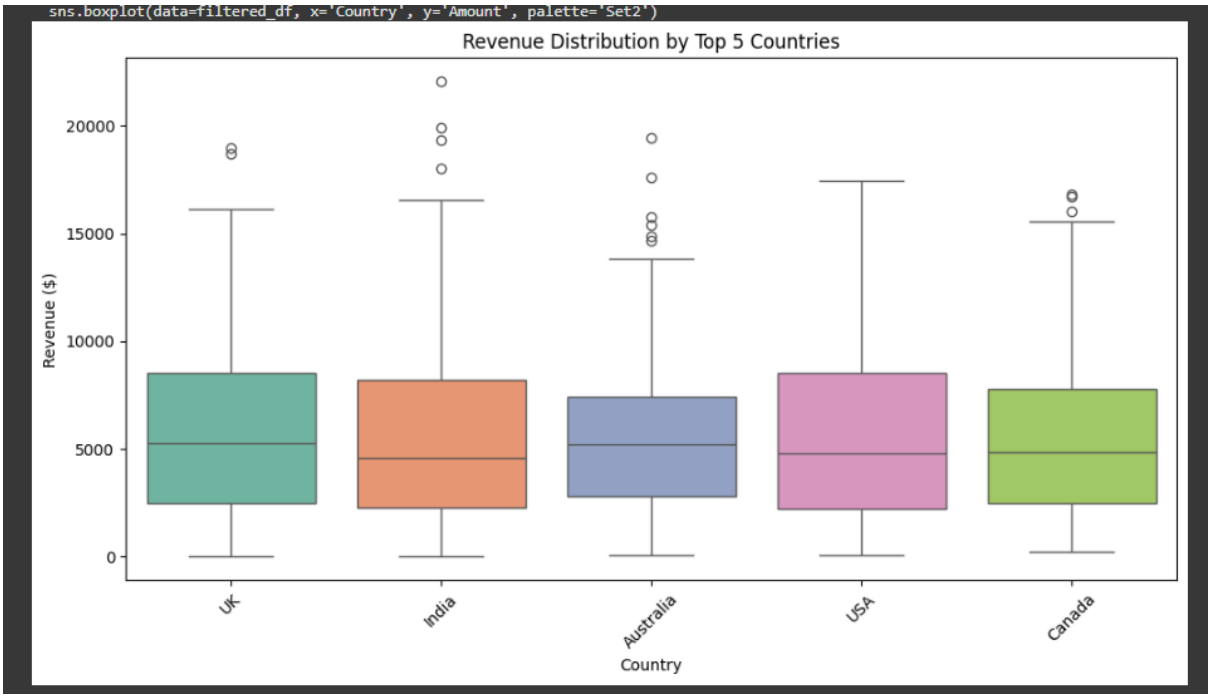
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('Processed_Chocolate_Sales.csv')

# Filter top 5 countries by total revenue
top_countries = df.groupby('Country')['Amount'].sum().nlargest(5).index.tolist()
filtered_df = df[df['Country'].isin(top_countries)]

# Box Plot
plt.figure(figsize=(12, 6))
sns.boxplot(data=filtered_df, x='Country', y='Amount', palette='Set2')
plt.title('Revenue Distribution by Top 5 Countries')
plt.xlabel('Country')
plt.ylabel('Revenue ($)')
plt.xticks(rotation=45)
plt.show()
```

Output:



Conclusion & Future Scope

Future Scope:

To deepen the analysis and drive sustained growth, the following avenues are recommended:

1. **Enhanced Data Collection**
    - **Incorporate customer demographics (age, gender) and purchase channels (online vs. in-store) to tailor marketing campaigns.**
    - **Add competitor pricing data and marketing spend to benchmark performance and identify competitive gaps.**
2. **Advanced Analytics**
    - **Apply machine learning models to forecast demand, optimize inventory, and predict seasonal trends.**
    - **Use clustering algorithms to segment markets or customers based on purchasing behavior.**
3. **Geographical Expansion**
    - **Explore untapped markets in Asia-Pacific or Europe by analyzing similar datasets from new regions.**
4. **Operational Integration**
    - **Merge sales data with supply chain metrics (e.g., production costs, delivery times) to identify bottlenecks and improve profitability.**
5. **Longitudinal Analysis**
    - **Extend the dataset to multiple years to assess annual trends, holiday impacts, and long-term product performance.**
6. **Sustainability Focus**

- **Analyze the impact of eco-friendly packaging or fair-trade certifications on sales to align with consumer preferences for ethical products.**

Conclusion:

The analysis of the Chocolate Sales.csv dataset uncovered actionable insights to drive strategic growth. Key findings highlight Australia and India as top-performing markets, fueled by strong demand for *Peanut Butter Cubes* and *85% Dark Bars*, while salespeople like *Van Tuxwell* excelled in revenue generation. Premium products (*99% Dark & Pure*, *Manuka Honey Choco*) dominated in the UK and USA, reflecting consumer preference for specialty items. Seasonal peaks in July-August (summer holidays) and January (post-holiday sales) emphasize opportunities for targeted promotions. Pricing strategies showed moderate alignment between sales value and volume, though outliers suggest room for optimizing bulk discounts or premium pricing. These insights advocate for reallocating resources to high-performing regions, tailoring marketing to seasonal trends, and enhancing premium product portfolios. Future efforts should expand data granularity (e.g., customer demographics, competitor pricing) and leverage predictive analytics to refine forecasts. By aligning operations with these findings, stakeholders can boost profitability, streamline inventory, and strengthen market positioning in the evolving chocolate industry.